Firmware Guide: how the engine-control software works

Date: 2026-10-11 **Covers:** `firmware/` as merged on master (commit `1149ecc`), plus the service link of 20 October (§16): bootloader, end-of-line record, underspeed warning. **Read with:** decisions log §11 (why it is built this way), §12 (the logs), §16 (the service link), and the specs in `docs/superpowers/specs/2026-10-1*`.

This guide explains what the firmware does and how, in the order a new engineer needs it: - what runs where - how one crank tooth and one millisecond are handled - the control loops - safety - CAN - the logs - every tuning knob - how it is tested - what the S32K148 port must provide - the service link: the laptop tool, the end-of-line record, the bootloader

---

1. What it is, and where it runs

The firmware is **plain C (C99) that touches no hardware.** It has: - no operating system - no memory allocation at run time - no waiting loops - no chip headers

Because of that, the same files run in three places:

Where	Who calls the firmware	State
A PC	`firmware/sim/run_sil.py`, through a Python model of the engine, generator and fuel rail	**Working: 15 scenarios pass**
NXP S32K148 evaluation board	interrupts and a main loop on the chip, with an Arduino faking the engine signals	Next: the dev-kit spec
Our own ECU board	the same, on our PCB	After layout

It is **bare-metal, not RTOS-based**, on purpose. The work is one 1 ms loop plus a crank-tooth interrupt. A scheduler would add stacks and priority problems without adding anything this ECU needs. The short, fixed overspeed path is also easier to prove without one. If a customer ever requires AUTOSAR OS, the 1 ms entry point becomes a 1 ms task and nothing inside the core changes.

2. Map of the code

File	Job
`core/ecu.h`	The whole interface: the `ecu_io` struct and the entry points. Start here.
`core/ecu.c`	The state machine, the 1 ms schedule, trip handling, the save policy and the watchdog
`core/crank.c`	Reads the 60-2 crank wheel and the cam: synchronisation, speed, angle

File	Job
`core/inject.c`	Turns fuel and angle into per-cylinder injection commands
`core/govern.c`	Speed governor: PI, setpoint ramp, droop
`core/rail.c`	Rail-pressure loop on the metering valve
`core/sensors.c`	Volts to engineering units, range faults
`core/safety.c`	The trip decision and the table of active faults
`core/mode.c`	The engine state machine and the guards that keep their own time (boost, EGR stall, water in fuel, underspeed, main relay), stepped by `ecu.c` (§17)
`core/model_glue.c`	`ECU_MODEL` builds: the same functions from the code generated out of `model/` (§17)
`gen/`	The C Embedded Coder generated from the Simulink/Stateflow models in `model/`. Committed; do not edit
`core/j1939.c`	Every CAN frame the ECU sends, and request parsing
`core/nvm.c`	The persistent image: layout, checksum, choosing between the two copies
`core/log.c`	Writes into the image: events, counters, fault memory, trip recorder
`core/isotp.c`	ISO-TP framing for the service link on CAN channel 2
`core/link.c`	USB framing for the service link: sync byte, length, CRC-16
`core/service.c`	The UDS core both sides share: sessions, seed-key, tester present, reset
`core/eol.c`	The end-of-line record: defaults, checksum, trim limits, picking a copy
`boot/boot.c`	The bootloader: run the application or stay, erase, download, check
`tools/ecu_service.py`	The laptop service tool (§16)
`sim/test_core.c`	101 unit tests of the C code
`sim/test_boot.c`	7 bootloader tests against a simulated flash
`sim/ecu.py`	Python wrapper around the compiled core; `Unit`, a simulated board for the tool
`sim/engine.py`	The engine model
`sim/run_sil.py`	The 16 scenarios
`sim/test_service.py`	The service tool end to end against `Unit`

All of the core's state lives in `static` variables inside `ecu.c` and `mode.c`. There is one ECU per program, and `ecu_init` resets it.

3. The interface

3.1 The `ecu_io` struct

All the firmware sees of the outside world is this one struct. The platform writes the inputs and reads the outputs.

Group	Fields	Meaning
Inputs	`v_clt`, `v_oil`, `v_rail`, `v_boost`, `v_egr`	Sensor voltages at the ADC pins (V)
	`v_5v`	Measured 5 V sensor supply; the ratiometric sensors are corrected by it
	`v_bat`	Battery voltage, recorded in fault snapshots
	`ignition`, `droop_sw`	Switch inputs, 0 or 1
	`reset_cause`	Set once before loading: 0 power-on, 1 watchdog, 2 other
Outputs	`inj[3]`	Per cylinder: start time, duration (µs) and a sequence number `seq`
	`meter_duty`	Metering-valve duty, 0 to 1
	`wdg`	The watchdog pin level
	`can[16]`, `can_n`, `can_drops`	CAN frames to send; the platform sends `can_n` of them, then zeroes it
	`nvm_save`, `nvm_slot`	"Write the image to this copy now", cleared by the platform when done
Status	`state`, `rpm`, `fuel_mg`, `rail_bar`, `clt_c`, `oil_kpa`, `fault_n`, `synced`	For display and tests
Service link (§16)	`boot_ver`	Input: the bootloader's version, for UDS 0xF180
	`usb_in[64]`, `usb_in_n`	Bytes received on USB; the next `ecu_tick_1ms` consumes them and zeroes the count (at tick priority: they change state the interrupts share)
	`usb_out[512]`, `usb_out_n`	Bytes to send on USB; the platform sends them, then zeroes the count
	`can2[16]`, `can2_n`	Frames to send on CAN channel 2, as `can[]`
	`eol_save`	"Write the end-of-line record to copy a, then copy b", cleared when done. The image does not change while it is set; a later change asks again once it is cleared
	`boot_req`, `reset_req`	Set the stay flag and reset into the bootloader; a plain reset
	`session`	Status: the UDS session (1 default, 2 programming, 3 extended)
Bosch pinout (ABI 2)	`stop_sw`, `wif`, `diag_sw`	Inputs, logical (the platform applies each contact's polarity): pin 25 stop, pin 21 water in fuel, pin 14 diagnosis request
	`v_sync`	Input: pin 09 volts, the GCU's sync. Read, no function yet
	`main_relay`, `afterrun`	Output: hold the main relay (pin 50), the board's own supply; status: holding it after ignition off
Boost and EGR guards (ABI 3)	`v_boost_rail`	Input: BOOST_100V volts. 0 means not measured (production has no channel; the prototype reads `BOOST_MON`)
	`i_egr`	Input: EGR bridge current (A)
	`boost_dis`	Output: 1 = stop the boost converter (`BOOST_DIS`, DIS/EN)
	`egr_off`	Output: 1 = the EGR bridge coasts (`EGR_EN` low)

Injection is given as commands, not pin levels. For each cylinder the command is "start at time T, for D microseconds". The chip's timer output-compare runs that directly. Every change to a command, including a cancel (duration 0), increments `seq`, so the platform only needs to watch `seq`. A cancel stops a command that hasn't started. An injection already open runs to its end.

3.2 Entry points

Function	Called by	How often	Does
`ecu_init`	start-up	once	Resets everything
`ecu_nvm_load`	start-up, after init	once	Loads the newer good copy of the image, logs the power-up
`ecu_crank_edge`	crank-sensor interrupt	every tooth (about 1450/s at 1500 rpm)	Decodes the tooth, **checks overspeed**, schedules injection
`ecu_cam_edge`	cam-sensor interrupt	once per 2 revolutions	Marks which revolution has cylinder 1 firing
`ecu_tick_1ms`	1 ms timer interrupt	1000/s	Everything slow (§5)
`ecu_can_rx`	CAN receive	per frame	Answers J1939 requests (§9)
`ecu_background`	main loop	as often as it likes	Saves, and checks in for the watchdog
`ecu_nvm_image`	main loop	when `nvm_save` is set	Gives the sealed image to write
`ecu_eol_load`	start-up, after init	once	Loads the good copy of the end-of-line record; neither good means SPN 628 FMI 13 and untrimmed inputs
`ecu_eol_image`	main loop	when `eol_save` is set	Gives the record to write
`ecu_can2_rx`	CAN channel 2 receive	per frame	The service link (only ID 0x7E0)
`ecu_abi_version`, `ecu_io_size`	the simulator, on load	once	The interface version and `sizeof(ecu_io)`; `ecusim` refuses a firmware library whose values differ from its own. Bump `ECU_ABI_VERSION` with any change to `ecu_io` or an entry point

One rule for the port: `ecu_crank_edge`, `ecu_cam_edge`, `ecu_tick_1ms`, `ecu_can_rx` and `ecu_can2_rx` share data and must not interrupt each other. Give their interrupts the same priority. `ecu_background` runs at the lowest level and may be interrupted at any time.

4. One crank tooth

This is the fastest and most important path. At 1500 rpm a tooth arrives every 0.67 ms.

1. **Noise filter.** A tooth that arrives in under half the previous tooth's time is ignored. No engine accelerates that fast, so it can only be noise. A tooth arriving after more than 50 ms of silence restarts the decoder's

history.

2. **Gap and sync.** The wheel has 58 teeth and a gap where two are missing, so the gap shows up as a period more than 1.5× the last one. Two gaps exactly 58 teeth apart mean **synchronised**. Once synchronised, a gap in the wrong place, or no gap where one belongs, is **sync loss**.
3. **Phase.** The crank turns twice per engine cycle, and the cam pulse says which turn has cylinder 1 firing. Until a cam pulse has been seen, no fuel is injected. A cam pulse in the wrong turn stops injection until the next one agrees. It does not shut the engine down, because one noise spike shouldn't stop a genset.
4. **Speed.** rpm comes from the last 6 tooth periods, with the gap counted as its true 18°.
5. **Safety, on this very tooth.** If speed is above **1650 rpm**, the ECU shuts down here, inside the interrupt, and cancels every injection that hasn't started. If this tooth itself raised a sync fault (a lost tooth reads as a speed jump), it is diagnosed as a crank sensor fault, not overspeed.
6. **Injection.** Each cylinder starts injecting **11.5–16° before top dead centre**, from a table on fuel copied from the old ECU (11.5° at 6.5 mg, 15.9° at 17 mg, 14.2° from 26.5 mg up). Cylinder 1's TDC is 90° after the gap; cylinders 2 and 3 follow at 240° and 480°. The command is written on the last tooth at least **1° before** that angle, so the start time is never already past. The start time is extrapolated from the current tooth speed.

5. One millisecond

`ecu_tick_1ms` runs, in this order:

1. The ms counter, which wraps every hour (3 600 000) by design.
2. **Watchdog:** toggle the WDI pin every 5 ms (a falling edge every 10 ms), but **only if the main loop has checked in within 50 ms**.
3. Once a second: the counters (§10).
4. Ignition handling: the shutdown latch (§6), and the ignition-off event after 500 ms off.
5. Crank timeout check: 50 ms without a tooth while synchronised is sync loss.
6. Sensors: convert, range-check, update the fault table (§8).
7. Safety check (overspeed, sync loss, ignition off).
8. Governor (in RUN) and rail loop (in CRANK and RUN).
9. CAN messages due this millisecond (§9).
10. A trip-recorder sample every 20 ms.
11. The status fields.

6. The state machine

```
OFF ──ignition on──▶ WAIT ──first tooth──▶ CRANK ──phased and ≥ 400 rpm──▶ RUN
                                                          (any state)
            overspeed > 1650 rpm  ─┐
            sync loss while RUN   ─┼──▶ SHUTDOWN (latched)
            ignition off          ─┘
SHUTDOWN ──ignition off ≥ 500 ms with the engine stopped, then on──▶ OFF ──▶ WAIT
```

- **WAIT:** powered, ignition on, waiting for the engine to turn.
- **CRANK:**
- the starter is turning the engine
- fixed start fuel of 30 mg once phased
- a decoder fault just restarts synchronisation
- **RUN:**
- the governor owns the fuel
- on entry the speed setpoint starts at the current speed and ramps at 500 rpm/s to 1500, which stops the start from overshooting towards the trip
- **SHUTDOWN:**
- no injection
- metering valve closed
- fault codes active
- CAN keeps reporting, so the DSE sees the engine run down

Underspeed is a warning, not a state. In RUN, once the setpoint ramp has finished, speed below **1350 rpm for 3 s** raises SPN 190 FMI 18 in DM1 with the **amber** lamp only. Speed above **1400 rpm for 1 s** clears it. The engine keeps governing: the DSE's own under-frequency settings decide whether the set stops. 1350 rpm is also the load-step scenario's floor, so a normal load step never raises it. (The red and Protect lamps stay with the real shutdowns: overspeed, SPN 190 FMI 0, and sync loss.)

The latch clears only on a real ignition cycle: ignition off for at least 500 ms *with the engine stopped*, then on. A bouncing contact or a flicker while the engine still turns does nothing.

7. The control loops

Governor (`govern.c`): - PI on the speed error, output = fuel per injection, clamped to 0–45 mg. - Isochronous at 1500 rpm. - With the droop switch closed, the setpoint falls 4 % from no load to full load (34 mg, about 20 kW). - The integrator is clamped (anti-windup). - Gains: 0.5 mg/rpm and 2.0 mg/(rpm·s), tuned in simulation, which gives a 3.4 % dip (1500 to 1449 rpm) on a full 20 kW step.

Rail pressure (`rail.c`): - PI drives the metering valve to a **setpoint that follows fuel**: 827 bar at 6.5 mg rising to 1171 bar at 26.5 mg, from the old ECU's own table at 1500 rpm (held flat beyond the data). - The valve is **normally open** (`METER_NC 0`): no power, full flow; more duty, less flow; full duty stops it. The old ECU's recording shows exactly this (decisions log §14). The valve's part number is the final confirmation. Because losing the drive now means full flow, **the trip module has to act on the injector supply**, not the valve's: a hardware change, still open. - In SHUTDOWN the valve is commanded closed. - If the rail sensor fails, the valve is held at a fixed 30 % opening (duty 0.7 for a normally-open valve), so the engine keeps running.

Injection duration (`inject.c`): duration = 323 µs dead time + fuel ÷ (0.00191 × √rail pressure). Both constants are fitted to the old ECU's own energising times (20 473 samples, within ±6 µs). Rail pressure is floored at 100 bar, so a failed or zero reading can't produce an enormous pulse.

8. Sensors

Every input is corrected by the measured 5 V supply. If that supply reads outside 4.5–5.5 V, 5.0 V is used instead.

Sensor	Scaling (placeholder)	Valid volts	Default on fault	J1939 SPN
Coolant temperature	NTC 2.5 kΩ at 20 °C, β 3450, 2.49 kΩ pull-up	0.1–4.8 V	90 °C	110
Oil pressure	0.5–4.5 V = 0–1000 kPa	0.2–4.8 V	300 kPa	100
Rail pressure	0.5–4.5 V = 0–1800 bar	0.2–4.8 V	1000 bar	157
Boost pressure	0.5–4.5 V = 50–400 kPa	0.2–4.8 V	100 kPa	102
EGR position	0.5–4.5 V = 0–100 %	0.2–4.8 V	0 %	27

End-of-line trims. Each input, and the 5 V reference, is corrected as `v × gain + offset` before anything else, with the six gains and six offsets from the unit's end-of-line record (§16). The factory measures them; the limits are gain 0.9–1.1 and offset ±0.1 V. With no good record the trims are 1 and 0, and SPN 628 FMI 13 ("out of calibration") stays on DM1, so a unit that skipped the line says so on the DSE's screen.

Range faults: - Out of range for **100 ms** raises a fault: FMI 3 for too high (an open circuit), FMI 4 for too low (a short). - **From the first moment out of range, the last good value is held.** The 100 ms delay only delays the fault code; it never lets a bad value through. - While a channel is faulted, its default is used for control and "not available" (0xFF) is sent on CAN. - The coolant low limit is 0.1 V, not 0.2 V. 0.2 V is a real 129 °C, and a hot engine must not be mistaken for a shorted sensor.

9. CAN (J1939, source address 234)

Sent all the time:

Message	PGN	Every	Carries
EEC1	61444	20 ms	Engine speed (0.125 rpm/bit)
EFL/P1	65263	500 ms	Oil pressure
ET1	65262	1 s	Coolant temperature
DM1	65226	1 s	Active faults and lamps: red + Protect = shutdown fault, amber = sensor fault. The DSE shuts down on Protect and treats red as a warning, so a shutdown fault must set both
HOURS	65253	10 s	Engine hours (0.05 h/bit)

Sent on request (a J1939 request addressed to 234 or to everyone):

Request	PGN	Answer
DM2	65227	Stored (previously active) faults
DM3	65228	Clears stored faults; active faults are untouched
DM4	65229	Fault snapshots: speed, load, coolant, then rail, oil, battery, state
HOURS	65253	Engine hours

Long messages. A message longer than 8 bytes goes as a broadcast sequence: one announcement, then packets 50 ms apart. Three rules protect it: - Only one sequence is in flight at a time. - The active fault list (DM1) always goes before DM2/DM4, so a service tool can't starve the controller of faults. - A sequence is sent whole or not at all: no announcement, no packets, and a packet that meets a full queue is retried 1 ms later.

The exact set of messages the DSE4522 expects is still open item U4.

10. The logs

Everything below lives in one **persistent image** of 3036 bytes, which fits the S32K148's 4 KB emulated EEPROM.

Record	Holds
Counters	ECU powered seconds, engine hours, starts, trips by cause, watchdog resets, time at each quarter of rated load
Fault memory	16 faults; each with an occurrence count, first and last time seen, and a **snapshot taken the first time**: speed, load, fuel, rail pressure, coolant, oil, battery, state
Event log	The last 64: power-up (with cause), ignition on/off, crank, run, shutdown (with reason), fault set/cleared, stored-fault clear, image reset
Trip recorder	The last 2 overspeed or crank-signal trips: speed, fuel, rail pressure and state every 20 ms, from 2 s before to 1 s after

Saving: - The platform keeps **two copies** and writes them alternately. Each copy carries a sequence number and a CRC-32 over the whole image. At power-up the newer good copy wins, so a power cut during a write costs one save, never the history. - An image from a different layout version is never misread. It is treated as blank, and an "image

reset" event is logged. - At power-up, every stored fault is marked previously active until it is seen again. - Saves are requested only by `ecu_background`, never from an interrupt, on any of these: - a fault's first occurrence - each start - entering shutdown - each hour of running - a watchdog-reset boot - a trip (1 s after it, once the recording is complete) - Saves are capped at one per 10 s, except a trip. That is about 30 a day, roughly 110 000 per copy over 20 years. - A fault's set and clear are logged once per ignition cycle, so a loose connector can't flood the event log.

Reading: stored faults, snapshots and engine hours over J1939 (§9). The event log and trip recorder need the laptop service tool from the dev-kit spec. Until then, the simulator decodes them from the image (`decode()` in `run_sil.py`).

11. Safety, in layers

Layer	Catches	Where
Overspeed check on every tooth	Runaway: trips at 1650 rpm, under the DSE's own 1680/1710 rpm trips	`ecu_crank_edge`
Sync-loss trip	Lost crank signal (50 ms) or a wheel that no longer makes sense	`crank.c` , `ecu.c`
Injection cancel	Any command not yet started when a trip happens	`inject_cancel`
Metering valve closed in SHUTDOWN	Rail pressure falls after a trip	`rail.h` `METER_CLOSED`
Underspeed warning	A set held below 1350 rpm for 3 s (overload, a fuel fault): SPN 190 FMI 18, amber. **No shutdown**: the DSE decides	`ecu.c` `underspeed()`
Boost rail warnings	Held over 104 V for 2 s (the hardware zener clamping, below the 106 V trip): amber SPN 520192 FMI 16. Under 80 V for 0.5 s while cranking or running (a dead boost): amber FMI 18. Both clear after as long the other way	`mode.c` `boostm_step`
Boost overvoltage	Rail over 106 V for 20 ms: stop the boost, SHUTDOWN, red SPN 520192 FMI 0. The boost is also off in SHUTDOWN. A 110 V zener on DIS/EN backs it in hardware (105–119 V)	`ecu.c` `boost_guard()` ; injector sheet
EGR stall	Over 5 A for 1.5 s: the bridge coasts 5 s, amber SPN 2791 FMI 6; the third stall holds it off to the next ignition cycle. **No shutdown**	`ecu.c` `egr_guard()`
Window watchdog (TPS3850)	A hung main loop: the pulses stop and the chip resets the processor; the next power-up logs it	`ecu_tick_1ms` , `ecu_background`
Independent trip module	Everything above failing: it cuts the metering valve's supply from outside the board	Hardware, spec §3

The firmware is the first line, not the only one. The trip module doesn't depend on the firmware at all.

12. Every tuning knob

Each is one `#define`. *Assumed* means a guess to be replaced with the real engine's figure. *Calibration* means tuned in simulation and to be re-tuned on the engine.

Knob	File	Value	Status
`CRANK_TDC1_DEG`	crank.h	90° after the gap	assumed (60-2 wheel assumed; **firing order 1-2-3 confirmed** 13 Oct 2026)
`CRANK_TIMEOUT_US`	crank.h	50 ms	design
`OVERSPEED_RPM`	safety.h	1650	design
`GOV_RPM_SET` , `GOV_DROOP`	govern.h	1500, 4 %	design
`GOV_KP` , `GOV_KI`	govern.h	0.5 mg/rpm, 2.0 mg/(rpm·s)	calibration
`GOV_RAMP_RPM_S`	govern.h	500 rpm/s	calibration
`FUEL_MAX_MG` , `FUEL_RATED_MG`	govern.h	45, 34 mg	assumed
`FUEL_START_MG`	inject.h	30 mg	calibration
SOI table `SOI_Q` / `SOI_D`	inject.c	11.5–16° BTDC on fuel	measured (old ECU, 1500 rpm)
`INJ_DEAD_US` , `INJ_K`	inject.h	323 µs, 0.00191	measured (old ECU)
`METER_NC`	rail.h	0 (normally open)	measured behaviour; confirm with the part number
Rail table `RAIL_Q` / `RAIL_P`	rail.c	827–1171 bar on fuel	measured (old ECU, 1500 rpm)
`RAIL_KP` , `RAIL_KI`	rail.h	0.002, 0.01	calibration
`RAIL_OPEN_LIMP`	rail.h	30 % opening	calibration
Sensor tables	sensors.c	§8	assumed until each front-end is measured
`RUN_RPM`	ecu.c	400	design
`IGN_OFF_MS`	ecu.h	500 ms	design
`BOOST_OV_V` , `BOOST_OV_MS`	ecu.h	106 V for 20 ms	design (rail 99.7 V ±2 %)
`BOOST_HIGH_V` , `BOOST_HIGH_MS`	ecu.h	104 V for 2 s	design (the zener's 105 V low corner)
`BOOST_LOW_V` , `BOOST_LOW_MS`	ecu.h	80 V for 0.5 s, cranking or running	design
`EGR_STALL_A` , `EGR_STALL_MS`	ecu.h	5 A for 1.5 s	assumed (stall 6.4 A; the running current is unmeasured)
`EGR_COOL_MS` , `EGR_STALL_TRIES`	ecu.h	5 s, 3	design
`SAVE_GAP_S`	ecu.h	10 s	design (flash wear)
`UNDERSPEED_RPM` , `UNDERSPEED_MS`	safety.h	1350 rpm for 3 s	design (90 % of 1500)
`UNDERSPEED_CLEAR_RPM` , `UNDERSPEED_CLEAR_MS`	safety.h	1400 rpm for 1 s	design
`SEC_SECRET`	service.h	16 bytes, overridable per build	**set per product before shipping**
`SVC_SESSION_MS` , `SVC_LOCK_MS`	service.h	5 s, 10 s	design
`EOL_GAIN_MIN/MAX` , `EOL_OFFSET_MAX`	eol.h	0.9–1.1, ±0.1 V	design

The engine model in `sim/engine.py` has its own assumed constants, also marked. When real engine data arrives, change the constant, re-run the scenarios, and they say at once whether the control still holds.

13. How it is tested

```
make -C firmware/sim test                        # 101 unit tests and 7 bootloader tests, a few s
.venv/bin/python firmware/sim/run_sil.py         # 16 scenarios, about 2-3 min
.venv/bin/python firmware/sim/run_sil.py rail    # scenarios whose name contains "rail"
.venv/bin/python firmware/sim/test_service.py    # the service tool end to end, about 10 s
```

#	Scenario	Proves
1	Start	in sync within 2 revolutions; 1500 rpm by 5 s; peak under 1600
2	Load step 0 → 20 kW	dip stays above 1350 rpm; back within ±15 rpm in 5 s; no underspeed warning
3	Load rejection 20 → 0 kW	peak stays under the trip
4	Droop switch	speed falls 4 ± 1 % at full load
5	Stuck injector (runaway)	shuts down on the very tooth it first reads > 1650 rpm; no injection after; rail pressure falls
6	Crank signal lost	shutdown within 100 ms, crank-sensor fault
7	Ignition off	shutdown with no fault code
8	Coolant sensor unplugged	fault within 1.2 s; engine keeps running
9	CAN timing and content	every message on time; speed within 0.5 %
10	Rail pressure	within 5 % of the setpoint for the fuel delivered
11	Watchdog	pulses every 10 ms, right to the end of the run
12	Runaway, then power cycle	fault, snapshot, event and trip recording all survive
13	Power cut mid-save	the previous save loads
14	Stored faults over CAN	DM2 reads them; DM3 clears them and leaves active ones
15	One lost crank tooth	logged as a crank-sensor fault, not overspeed
16	Overload, 30 kW	settles near 1190 rpm; SPN 190 FMI 18 within 5 s, amber only; never shuts down

Scenario 16 relies on one model assumption: the alternator's voltage regulator rolls the voltage off below 47 Hz (`AVR_KNEE_RPM`), so the load's power falls with speed and an overloaded set settles instead of stalling. With a pure constant-power load every overload stalls in under 3 s, faster than the warning.

What a pass means, and what it doesn't. The scenarios prove the logic and the safety behaviour. They don't prove the tuning on a real engine. The engine model uses the firmware's own injector and sensor formulas, and it has no electrical noise. Two independent expert reviews, each reproducing its findings, were run against the code (decisions log §11 and §12).

14. What the S32K148 port must provide

The core needs nothing else. The platform layer must:

1. **Crank input:** input capture on the crank sensor; call `ecu_crank_edge` with the capture time in µs (a free-running 32-bit counter). Same for the cam.
2. **1 ms timer:** call `ecu_tick_1ms` with the same µs clock.
3. **Injection:** after each call, if `inj[k].seq` changed, load `start_us` / `dur_us` into that cylinder's output-compare channels. Duration 0 means disarm, unless the injection has already started.
4. **Metering valve:** drive the PWM from `meter_duty`.
5. **CAN:** send `can[0..can_n)` and zero `can_n`. Pass received frames to `ecu_can_rx`.
6. **Watchdog:** copy `wdg` to the TPS3850's WDI pin. Strap its window (SET0/SET1, CWD) to accept a 5–20 ms kick period.
7. **Storage:** keep two copies of the image. When `nvm_save` is set, write `ecu_nvm_image()` to copy `nvm_slot`, **without blocking the main loop for more than 50 ms** (or the watchdog resets the chip), then clear `nvm_save`. At start-up pass both copies to `ecu_nvm_load`, with `reset_cause` from the chip's reset status.
8. **Main loop:** call `ecu_background` continuously.
9. **Priorities:** the edge, tick and CAN-receive interrupts at one priority. Mask them during the image copy inside `ecu_background`.
10. **Service link:** pass CAN channel 2 frames to `ecu_can2_rx` (at the tick's priority); send `can2[0..can2_n)`. On the prototype, copy USB virtual-serial bytes into `usb_in` (up to 64 per ms; the tick takes them) and send `usb_out`.
11. **End-of-line record:** two copies in their own sector (S32K148: a D-flash sector; STM32G474: the last page). Pass both to `ecu_eol_load` at start-up. When `eol_save` is set, write `ecu_eol_image()` to copy a, then copy b, then clear it. Pass the bootloader's version in `boot_ver`.
12. **Resets:** on `reset_req`, a software reset. On `boot_req`, set the stay flag (one RAM word the reset does not clear) and reset.
13. **The bootloader's host:** `flash_erase`, `flash_write` (whole 8-byte words), `flash_read` over the application area only, the stay flag (read and clear), and `jump_to_app` (stack pointer and vector table from the application's base, then branch). §16 has the rest.

15. Known limits and open items

Needs the machine or a supplier: - the metering valve's part number (behaviour already measured: normally open) - crank wheel (firing order 1-2-3 is confirmed) - the sensor tables

Needs the real engine: - governor and rail tuning; the old ECU dips 9.8 % on a load step, so our simulated 3.4 % is likely optimistic - start fuel, and timing and rail pressure while cranking (the recording starts with the engine already running) - timing and rail tables above 26.5 mg, and at speeds other than 1500 rpm - pilot and post injections (the old ECU uses none in this mode)

Needs U4 (the DSE's exact expectations): - the exact J1939 message set - whether DM3 needs an acknowledgement

Dev-kit spec: - flash choice for about 110 000 writes per copy - the service tool's reading of the event log and trip recorder (the record and flashing are done, §16) - the chip layers of the service link: STM32G474 and S32K148 flash, CAN, USB and reset drivers (§14 items 10–13) - MISRA C checks - governor performance graded against ISO 8528-5 - a failure analysis of the overspeed path

Known minor items: - An extra crank tooth from a noise spike still logs as overspeed. The engine stops correctly either way. - A faulted sensor's snapshot stores its default value rather than "not available". - DM1 always reports an occurrence count of 1.

16. The service link

One protocol, a subset of UDS (ISO 14229), on two links: **USB** on the prototype for quick iterations, **CAN channel 2** in production (ECU pins 86/87, off the DSE's J1939 bus). The application and the bootloader share `core/service.c`; only the framing differs.

Link	Framing
CAN channel 2	ISO-TP, 11-bit: request `0x7E0`, response `0x7E8`, 1000 kbit/s (Bosch's application CAN, pins 87/86), frames padded with `0xAA`, block size 0, STmin 0, up to 4095 bytes
USB (virtual serial)	`0x55`, length (2 bytes LE), payload, CRC-16/CCITT of the payload (2 bytes LE). A bad CRC or a length over 4095 drops the message

Services:

Service	Application	Bootloader
`0x10` session	default `01`, extended `03`; programming `02` (unlocked) resets into the bootloader	default, extended, programming
`0x27` security	seed `01`, key `02`; not in the default session	same
`0x3E` tester present	keeps a session alive (`3E 80`: no reply)	same
`0x11` reset	`01`, engine stopped only	`01`: runs the image if it is good
`0x22` read	`F18C` serial, `F191` hardware revision, `F18B` build date, `F195` software version, `F180` boot version, `0100` trims, `0101` station, passed, locked	`F180`, `F195` (from the image header), `FD00` the application area

Service	Application	Bootloader
`0x2E` write	the record's IDs: extended, unlocked, engine stopped, record not locked	—
`0x31` routine	`F000` lock the record, `F001` unlock it (logged)	`FF00` erase (address, length), `FF01` check (CRC, version, hardware range)
`0x34` / `0x36` / `0x37`	—	download, 1024-byte blocks, exit

Refusals are `7F, service, reason`: `22` conditions (the engine turning, or the record locked), `33` security, `35` wrong key, `36` too many wrong keys, `31` out of range, `24` wrong sequence, `13` wrong length, `72` programming failure, `7F` wrong session.

Rules: - **Engine stopped** means OFF, WAIT or SHUTDOWN and no crank tooth for 1 s (a saturating counter, so it holds however long the set rests). Extended and programming sessions, and an ECU reset, are refused otherwise, and an open session ends at the first tooth. - A non-default session ends 5 s after the last message. - **Seed-key:** the key is the low 32 bits of **SipHash-2-4** of the seed (4 bytes LE), keyed with the 16-byte `SEC_SECRET`. Three wrong keys lock security access for 10 s. A sniffed seed and key reveal nothing about other seeds (the first design's CRC-32 key did: it is affine). It still stops casual writes, not a determined attacker: anyone who reads one unit's flash has the secret for all.

The end-of-line record (`core/eol.h`, 84 bytes): magic `0xE01D`, version, serial (16 ASCII), hardware revision, build date (yyyymmdd), six gains and six offsets (coolant, oil, rail, boost, EGR, 5 V reference), station, passed, locked, CRC-32. Two copies apart from the logs; copy a is written first, so when both are good copy a is the newer. **Only a record stamped "passed" counts:** until the station writes that stamp, SPN 628 FMI 13 stays on, so a station that stops after the serial ships no silent unit. Once locked (`31 01 F000`) every write is refused until `31 01 F001`.

The bootloader (`boot/boot.c`, 32 KiB at the bottom of flash): - At reset: if the stay flag is set, stay. Otherwise run the application only if its header is good, its CRC matches, and the end-of-line record's hardware revision is inside the header's range. - Programming: programming session, unlock, erase, download to the application base + 64, transfer, exit, then the check routine. **The header is written last**, after the CRC of what is in flash matches the tool's, and its magic is the very last write. A power cut anywhere before leaves no valid header, so the next reset stays in the bootloader and the update is simply run again (`test_boot.c` cuts the power at every single write). - It never erases or writes outside the application area. While it runs the engine cannot: no

injection, the metering valve at its safe state, the gates held off by their pull-downs; the watchdog is still kicked. - No rollback: a bad image that passes its CRC is fixed by flashing the previous one.

The laptop tool:

```
T=firmware/tools/ecu_service.py; K=--secret key.bin      # key.bin: the 16 bytes of SEC_SECRET
.venv/bin/python $T --link usb:/dev/tty.usbmodem1 $K info
.venv/bin/python $T --link can:socketcan:can0 $K eol-read
.venv/bin/python $T --link usb:/dev/tty.usbmodem1 $K eol-write --serial SN0042 \
    --hw-rev 0x0100 --date 20261020 --trims trims.json --station 3 --passed
.venv/bin/python $T --link usb:/dev/tty.usbmodem1 $K eol-lock          # eol-unlock for a rework
.venv/bin/python $T --link usb:/dev/tty.usbmodem1 $K flash app.bin \
    --sw-ver 0x00010002 --hw-min 0x0100 --hw-max 0x01FF
.venv/bin/python $T --link sim $K info                                 # the simulated board
```

`trims.json` is `{"gain": [6 values], "offset": [6 values]}` in the order above. `pyserial` (USB) and `python-can` with `can-isotp` (CAN) are needed only by their links. A 256 KiB image takes about 10 s over CAN plus the erase; USB is faster.

Not yet: the chip layers (§14 items 10–13). The prototype is first flashed over SWD (J301), as any new board is.

17. The Simulink and Stateflow models (13 November 2026)

The control and state logic also exists as models, in `model/`, and Embedded Coder generates C from them into `gen/`. Building with `ECU_MODEL` defined puts that generated code in place of the hand-written modules, behind the same functions:

Function(s)	Model	Kind
`mode_cmd`	`ecu_mode`	Stateflow: OFF, WAIT, CRANK, RUN, SHUTDOWN
`boostg_*`, `boostm_*`, `egrg_*`, `wifg_*`, `underg_*`, `relay_*`	`ecu_boostg`, `ecu_boostm`, `ecu_egrg`, `ecu_wifg`, `ecu_underg`, `ecu_relay`	Stateflow
`gov_reset`, `gov_step`	`ecu_gov`	Simulink
`rail_reset`, `rail_step`, `rail_set_bar`	`ecu_rail`, `ecu_railset`	Simulink
`inject_soi_btdc`, `inject_dur_us`	`ecu_soi`, `ecu_injdur`	Simulink
`safety_check`	`ecu_safety`	Simulink

The state machine is driven by commands, because both the crank-tooth interrupt and the 1 ms tick change it: `ecu.c` sends TICK (every 1 ms, with the ignition and speed), TOOTH, RUN_OK, TRIP or RESET, and the chart answers with the new state. Since 14 November the chart owns its conditions: the ignition for OFF → WAIT, and the ignition cycle that ends SHUTDOWN (low for `IGN_OFF_MS` with the engine stopped, then on). When the chart lands in OFF that way, `ecu.c` resets

everything else. `ecu.c` keeps everything around it (events, the governor reset on RUN entry, counters, DTCs), and `gov_setpoint` stays hand-written in both builds.

Checking everything at once: `./verify.sh` at the top of the repository runs the circuits, schematics, both firmware builds, the back-to-back test, both sets of scenarios and the service tool, and names whatever failed. `--boards` adds the KiCad board checks, `--matlab` the Simulink closed loop. Run it before every commit that touches `firmware/`, `model/` or `hw/`.

Building it: `make test-model` (unit tests) and `build/libecu_model.*` (the scenarios, with `ECU_LIB=libecu_model`) in `sim/`. On Windows, call `zig.exe` from the venv directly (`.venv/Lib/site-packages/ziglang/zig.exe cc`): `python -m ziglang` swallows the compiler's error messages.

The evidence it is the same firmware: every unit test and all 16 scenarios pass on both builds; `model/test/run_b2b.sh` runs each generated model against its hand-written original on millions of random inputs, and they agree exactly, apart from one or two units in the last place in the two tables (Simulink orders its interpolation arithmetic differently from `table.h`). See `model/README.md`.

For the target: the generated code is single-precision C99 for an ARM Cortex-M. With the whole core it compiles warning-free for the Cortex-M4F (`zig cc -target thumb-linux-musleabihf -mcpu=cortex_m4+vfp4d16sp -mfloat-abi=hard -DECU_MODEL`, compile only), and the governor comes out as native `vadd.f32` / `vmul.f32` / `vdiv.f32` with no double-precision code. It is flashed as part of the firmware image. The chip layers (§14) are still what stands between either build and the board.

Which is the source of truth: for now, both. The hand-written modules stay the reference build until the generated build has run on the engine. A change goes into both: the model's script in `model/` (constants come from the C headers automatically), then `build_all` and `gen_code`, then the C reference, then both test runs.